

Let me add something about the use of the Greek letters. Look at the following two examples:

```
ylab=expression("Conductivity ("mu"S/cm)")
ylab=expression("Conductivity ("mu"S/cm)")
```

The first is preferable if the plot will be exported to PDF and the second, if a space is added after "(", if it is in PNG. With some limitations, it's also possible to directly use the 'micro' symbol, but probably the best thing is to use the unicode string "u00b5". I think that the use of a more general technique could be better than this.

Linear fit weighted and printed with Markdown

A weighted model is very widely used in some applications (for example, in pharmacokinetics studies). With a few lines of code I can define all the models I need. The most important aspect of this topic is the calculation of the percentage accuracy for the back-calculated concentrations, as reported in the following code:

```
# fit<-lm(y~x) # Linear fit simple
```

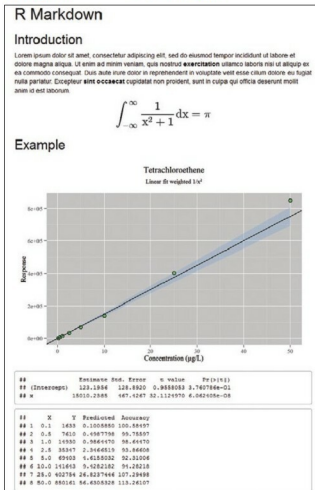


Figure 6: R Markdown example

```
fit<-lm(y~x,weight=1/x^2) # Linear fit weighted
a=coef(fit)[2]
b=coef(fit)[1]
# fit<-lm(y~0+x) # Linear fit through origin
# a=coef(fit)[1]
# b=0
summary(fit)$coef
Predicted=(y-b)/a
Accuracy=(Predicted*100)/x
cbind(data,Predicted,Accuracy)
```

The table that is made with `cbind`, in which the data is present, the x values back-calculated (usually the concentration is reported on the x axis) and its percentage accuracy, are all very useful. For more theory, see *Reference 7* given at the end of the article. Printing a notebook-like document is possible with the R Markdown package (though other options exist). Generally speaking, using my preferred text editor, I can write a document called, for instance, *example.Rmd* which contains text and code; then R is used as a compiler only. If Pandoc (<http://www.pandoc.org>) is installed, typing `rmarkdown::render("example.Rmd")` from the R console, produces the HTML file with the text formatted according to the Markdown syntax, a plot and the results of some calculations (Figure 6). It's very easy, so issue the following commands:

```
# Title

## Section 1

$$your-mathematical-formula$$

## Section 2

```{r,message=F,echo=F}
...
ggplot(data.frame(x=x,y=y),aes(x=x,y=y))+
 geom_ribbon(aes(y=p["fit"],ymin=p["lwr"],ymax=p["upr"],fill="
confidence"),alpha=0.1)+
 scale_fill_manual(values="#1E90FF")+
 geom_point(shape=21,col="black",fill="#90EE90",size=3)+
 geom_abline(slope=a,intercept=b)+
 labs(x="Concentration (µg/L)",y="Response")+
 ggtitle(expression(atop("Tetrachloroethene",atop("Linear fit
weighted 1/x^2"))))+
 theme(legend.position="none")

...
...

```

The plot is built with *ggplot* and the confidence band is added. In this example, the options *message* (to display R messages) and *echo* (to display the code along with its results) are set to 'False'. More complex documents can be produced